



Ostbayerische Technische Hochschule Amberg-Weiden
Department of Electrical Engineering, Media and Computer Science

AUTONOMOUS ROBOT

Autonomous robot in Webots simulation

Students

Tran Thai Duc Hieu

Ammar Haider

Arman Moinahmed Pathan

Supervised by

Prof. Dr. Thomas Nierhoff

February 6, 2026

1 Approach

1.1 Overview

This project implements an autonomous navigation system for a mobile robot in a simulated Webots environment. The robot's task is to navigate from a blue pillar (starting point) to a yellow pillar (goal) while avoiding obstacles, particularly green (Poison) regions. The system operates in two phases: an exploration phase where the robot maps the environment and locates both pillars, followed by a final execution phase where it traverses the optimal path from start to goal.

The core strategy combines frontier-based exploration for systematic environment coverage with reactive obstacle avoidance, enabling the robot to handle both pre-mapped obstacles and dynamic environmental features detected during navigation.

1.2 Strategy overview

The autonomous navigation strategy is built on three core principles:

1. Continuous perception

Vision-based detection runs continuously in background threads, identifying three critical features in real-time:

- **Blue and yellow pillars:** Landmark detection using camera color segmentation and depth estimation
- **Red walls:** Obstacle boundaries requiring closure marking
- **Green carpet:** Toxic regions to be avoided at all costs

2. Frontier-based exploration

Rather than random exploration, the robot systematically explores by targeting frontier regions—the boundary between known free space and unknown territory. Frontier targets are prioritized by:

1. Proximity to detected pillars (highest priority)
2. Size of unexplored regions (explore large unknown areas first)
3. Random selection (fallback strategy)

3. Reactive path following

Global paths (computed via A*) are followed locally using Dynamic Window Approach (DWA), enabling responsive collision avoidance without replanning the entire path.

1.3 Technical approaches

1.3.1 Perception and sensor fusion

The robot integrates two complementary sensing modalities:

LiDAR-based occupancy grid mapping The LiDAR continuously scans the environment and builds a probabilistic map using log-odds occupancy mapping. Each scan updates the map as follows:

- **Intermediate cells** (from robot to obstacle): marked as free space with log-odds decreasing by a tuned negative weight
- **Endpoint cell** (obstacle): marked as occupied with log-odds increasing by a tuned positive weight
- **Log-odds clipping:** values bounded to prevent numerical overflow
- **Probabilistic conversion:** $P(\text{occupied}) = \frac{1}{1+e^{-\text{score}}}$

Configurable probability thresholds classify cells: high probability values indicate obstacles, low values indicate free space, and intermediate values remain unknown. This soft probabilistic approach allows the robot to gradually incorporate sensor data and adapt to changing environments.

Concurrent threading Both LiDAR and camera detection run in independent background threads with thread-safe communication:

- Detection results are stored in thread-safe signal variables
- Main control loop checks signals at each timestep
- Mutex locks prevent race conditions during sensor access

This architecture ensures perception runs continuously without blocking motion control, achieving responsive obstacle detection.

1.3.2 Coordinate transformation matrix

The system uses a 2D rotation and translation matrix to convert points from the robot’s local reference frame to the global world coordinate system. This transformation is essential for mapping LiDAR observations and projecting camera detections onto the occupancy grid.

Transformation formula Given a point $\mathbf{p}_{\text{local}} = [x_{\text{local}}, y_{\text{local}}]^T$ in the robot’s coordinate frame, the corresponding world coordinates $\mathbf{p}_{\text{world}} = [x_{\text{world}}, y_{\text{world}}]^T$ are computed as:

$$\mathbf{p}_{\text{world}} = \mathbf{R}(\theta) \cdot \mathbf{p}_{\text{local}} + \mathbf{t}$$

where:

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

and $\mathbf{t} = [x_{\text{robot}}, y_{\text{robot}}]^T$ is the robot’s current position, and θ is the robot’s heading (yaw angle) in radians.

Implementation For efficient batch processing of multiple points (e.g., LiDAR point clouds), NumPy vectorization is used:

```
points_rotated = points_local @ R.T # Matrix multiplication
points_world = points_rotated + [x_robot, y_robot] # Broadcasting
```

This transformation is applied to:

- LiDAR point clouds for obstacle mapping
- Green carpet pixel projections for hazard marking
- Column position estimates for landmark localization

1.3.3 LiDAR-based occupancy mapping

The robot builds a probabilistic occupancy grid map using log-odds representation combined with Bresenham’s line algorithm for ray tracing.

Log-odds update rule Each LiDAR measurement updates the occupancy probability of cells along the ray from the robot to the detected obstacle. The log-odds score $L(x, y)$ for cell (x, y) is updated as:

$$L(x, y) = \begin{cases} L(x, y) - \alpha_{\text{free}} & \text{if cell is on ray (free space)} \\ L(x, y) + \alpha_{\text{occ}} & \text{if cell is endpoint (obstacle)} \\ L(x, y) & \text{otherwise (unchanged)} \end{cases}$$

where α_{free} and α_{occ} are tunable update weights. The log-odds values are clipped to a bounded range to prevent numerical overflow and maintain computational stability.

Probability conversion The discrete occupancy state of each cell is determined by converting log-odds to probability using the sigmoid function:

$$P(\text{occupied} \mid x, y) = \frac{1}{1 + e^{-L(x,y)}}$$

Cells are classified as:

- **Obstacle** if $P > P_{\text{thresh_high}}$ (high confidence occupied)
- **Free space** if $P < P_{\text{thresh_low}}$ (high confidence empty)
- **Unknown** otherwise (insufficient evidence)

where $P_{\text{thresh_high}}$ and $P_{\text{thresh_low}}$ are configurable probability thresholds.

Ray tracing with Bresenham For each LiDAR point, Bresenham’s line algorithm efficiently identifies all grid cells intersected by the ray from robot position to obstacle endpoint. This ensures accurate free-space marking along sensor beams without expensive floating-point computation.

1.3.4 Green carpet detection and marking

The green area represents hazardous regions that must be avoided. The system detects, projects, and permanently marks these areas on the occupancy grid using camera-based vision and coordinate transformation.

Pixel-to-ground projection Detected green pixels are projected onto the ground plane ($Z=0$) using an inverse pinhole camera model combined with the robot’s extrinsic parameters.

Step 1: Normalize pixel coordinates

For each pixel (u, v) in the camera image:

$$x_{\text{norm}} = \frac{u - c_x}{f_x}, \quad y_{\text{norm}} = \frac{v - c_y}{f_y}$$

where (c_x, c_y) is the principal point (image center) and (f_x, f_y) are the focal lengths in pixels.

Step 2: Calculate forward distance

Assuming the camera is mounted at height H above the ground and looking forward with pitch angle ≈ 0 :

$$D = \frac{H}{y_{\text{norm}}}$$

where D is the forward distance from the camera to the ground point. Only pixels below the horizon ($y_{\text{norm}} > 0$) are valid.

Step 3: Compute local coordinates

The ground point’s position in the camera’s local frame (X-forward, Y-lateral):

$$X_{\text{cam}} = D, \quad Y_{\text{cam}} = -D \cdot x_{\text{norm}}$$

Step 4: Offset to robot frame

Account for camera mounting offset relative to robot center:

$$X_{\text{robot}} = X_{\text{cam}} + \Delta X, \quad Y_{\text{robot}} = Y_{\text{cam}} + \Delta Y$$

where $(\Delta X, \Delta Y)$ represents the camera’s physical offset from the robot’s center.

Step 5: Transform to world coordinates

Apply the rotation and translation matrix (Section 3.2.4) to convert robot-local coordinates to global map coordinates.

1.3.5 Frontier-based exploration

Frontier cells are the boundary between known free space and unknown regions. The algorithm performs the following steps:

1. **Frontier detection:** Scan the grid for all free cells adjacent to unknown cells
2. **Clustering:** Use breadth-first search (BFS) to group connected frontier pixels into cohesive regions
3. **Prioritization:** Select target frontier based on:
 - Proximity to detected pillars (top priority, within 10 pixels)
 - Size of frontier region (larger = higher priority)
 - Random selection among remaining frontiers
4. **Target selection:** Pick the frontier region's centroid as navigation target

This systematic approach ensures the robot explores the environment efficiently rather than wandering randomly.

1.3.6 A* global path planning

The A* pathfinding algorithm computes optimal collision-free paths through the occupancy grid from start to goal. The implementation uses adaptive obstacle inflation and path quality validation to ensure safe, traversable routes.

Adaptive inflation strategy To balance path safety with feasibility, the planner employs escalating inflation levels $[I_1, I_2, \dots, I_n]$ pixels, attempting each in sequence until a valid path is found:

1. **Highest inflation level:** Conservative safety margin, attempts first
2. **Medium inflation levels:** Moderate safety, tried if higher levels produce no path
3. **Lowest inflation level:** Minimal clearance, allows navigation through narrow corridors

The specific inflation values are configurable parameters that can be tuned based on the robot's physical dimensions and environment characteristics.

Map preprocessing Before pathfinding, the occupancy grid undergoes several preprocessing steps:

1. **Noise removal:** Isolated obstacle pixels are removed using connected component analysis
2. **Small component filtering:** Obstacle clusters smaller than a minimum size threshold are eliminated to ignore sensor noise
3. **Obstacle inflation:** Remaining obstacles are dilated by the current inflation level using morphological dilation with an elliptical kernel
4. **Protected cell restoration:** Special markers (green carpet regions, closure areas) are reapplied as hard obstacles after inflation to ensure they are never traversed

A* search algorithm The core A* implementation uses:

- **Heuristic function:** Euclidean distance $h(n) = \sqrt{(x_n - x_{\text{goal}})^2 + (y_n - y_{\text{goal}})^2}$
- **Cost function:** $g(n)$ accumulates actual distance traveled (1.0 for cardinal moves, $\sqrt{2} \approx 1.414$ for diagonal moves)
- **Movement model:** 8-directional connectivity (N, S, E, W, NE, NW, SE, SW)
- **Priority queue:** Min-heap ordered by $f(n) = g(n) + h(n)$ with tie-breaking counter for deterministic behavior

Path quality validation Each candidate path is validated before acceptance:

$$L_{\text{path}} = \sum_{i=1}^{N-1} \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2} \cdot r$$

where r is the grid resolution in meters per pixel. Paths are rejected if $L_{\text{path}} < L_{\text{min}}$, where L_{min} is a configurable minimum path length threshold. This forces the planner to retry with higher inflation and avoids unrealistic shortcuts through tight gaps.

Path smoothing Raw A* paths contain grid-aligned zigzag patterns. To produce smooth, drivable trajectories, the path is post-processed using B-spline interpolation:

1. Detect sharp corners (angle changes above a threshold) as key waypoints
2. Apply natural cubic spline fitting to waypoint coordinates
3. Resample the spline at higher density for smoother motion

This smoothing reduces abrupt direction changes and improves tracking performance for the local planner.

1.3.7 DWA local motion planning

The Dynamic Window Approach (DWA) generates velocity commands to follow planned waypoints while reactively avoiding obstacles detected in real-time. DWA operates as a local planner that bridges the gap between global A* paths and low-level motor control.

Dynamic window concept DWA samples velocity pairs (v, ω) from a discrete set of admissible velocities and simulates their short-term trajectories. The "dynamic window" refers to the subset of velocities reachable given the robot's current motion constraints (acceleration limits, maximum speeds).

In this implementation, the window is simplified to predefined velocity samples:

- **Linear velocities:** A discrete set of forward velocity samples from low to maximum speed
- **Angular velocities:** A discrete set of angular velocity samples including zero, positive (left turn), and negative (right turn) values

The number of candidate velocity pairs is determined by the product of linear and angular sample counts, all evaluated at each planning cycle.

Trajectory simulation and collision checking For each (v, ω) pair, the robot's future pose is predicted over a short time horizon using the differential drive kinematic model:

$$\begin{aligned} x_{t+\Delta t} &= x_t + v \cos(\theta_t) \cdot \Delta t \\ y_{t+\Delta t} &= y_t + v \sin(\theta_t) \cdot \Delta t \\ \theta_{t+\Delta t} &= \theta_t + \omega \cdot \Delta t \end{aligned}$$

where Δt is the simulation timestep.

The trajectory is simulated over a configurable prediction horizon (multiple timesteps), checking for collisions at each predicted position by querying the occupancy grid. Trajectories are rejected if:

- Any predicted cell contains an obstacle (probability above threshold)
- The predicted distance to target increases by more than a configurable tolerance (prevents moving away from goal)

Multi-objective scoring function Valid trajectories are scored using a weighted combination of three objectives:

$$\text{score}(v, \omega) = w_h \cdot S_{\text{heading}} + w_d \cdot S_{\text{distance}} + w_v \cdot S_{\text{velocity}}$$

where w_h , w_d , and w_v are configurable weights for heading alignment, distance to target, and forward velocity respectively. The weights are tuned such that heading and distance objectives dominate over speed, prioritizing accurate navigation over velocity.

Heading Score: Measures alignment with the direction to the target waypoint:

$$S_{\text{heading}} = \cos(\theta_{\text{error}})$$

where $\theta_{\text{error}} = \theta_{\text{target}} - \theta_{\text{predicted}}$ is the angular difference between the predicted heading and the direction pointing toward the waypoint. This score ranges from 1 (perfect alignment) to -1 (opposite direction).

Distance Score: Rewards getting closer to the target:

$$S_{\text{distance}} = 1 - \frac{d_{\text{predicted}}}{d_{\text{norm}}}$$

where $d_{\text{predicted}}$ is the Euclidean distance from the predicted pose to the target waypoint, and d_{norm} is a normalization factor based on typical waypoint distances.

Velocity Score: Encourages higher speeds when safe:

$$S_{\text{velocity}} = \frac{v}{v_{\text{max}}}$$

where v_{max} is the maximum linear velocity defined by the robot's capabilities and safety constraints.

Control output The velocity pair (v^*, ω^*) with the highest score is selected and converted to differential wheel speeds:

$$\omega_L = \frac{v^* - \frac{L}{2}\omega^*}{r}, \quad \omega_R = \frac{v^* + \frac{L}{2}\omega^*}{r}$$

where L is the axle length (wheelbase) and r is the wheel radius, both determined by the robot's physical design.

2 Results

Table 1 shows the navigation completion times across all five maze configurations. Times are measured from the start of the exploration phase until the robot reaches the yellow pillar.

Table 1: Navigation Times for Each Maze Configuration

Map	Start → Yellow	Start → Blue	Blue → Yellow	Total
1	00:00 → 00:33	00:00 → 01:19	01:19 → 01:44	01:44
2	00:00 → 00:05	00:00 → 01:53	01:53 → 02:39	02:39
3	00:00 → 01:02	00:00 → 01:29	01:29 → 01:46	01:46
4	00:00 → 00:05	00:00 → 00:53	00:53 → 01:26	01:26
5	00:00 → 00:09	00:00 → 01:12	01:12 → 01:30	01:30

3 Declaration of AI Usage

In accordance with academic integrity guidelines, we hereby declare the use of AI-assisted tools during the development of this project.

3.1 Tools Used

- **GitHub Copilot** (integrated in Visual Studio Code)
- **ChatGPT** (OpenAI)

3.2 Purpose and Scope of Use

AI tools were used in the following capacities:

1. **Code assistance:** Autocompletion, syntax suggestions, and debugging support during the implementation of the robot controller, pathfinding algorithms, and sensor processing modules.
2. **Report writing:** Assistance with structuring, phrasing, and formatting sections of this report, including mathematical notation in $\text{\LaTeX}$.
3. **Problem solving:** Consultation for algorithmic design decisions such as log-odds mapping, DWA parameter tuning, and coordinate transformation formulations.

3.3 Declaration

All AI-generated suggestions were reviewed, understood, and adapted by the authors. The overall system design, architecture decisions, parameter tuning, testing, and final implementation are the original work of the project team. AI tools served as assistive aids and did not autonomously produce any final deliverable.

A Supplementary Figures

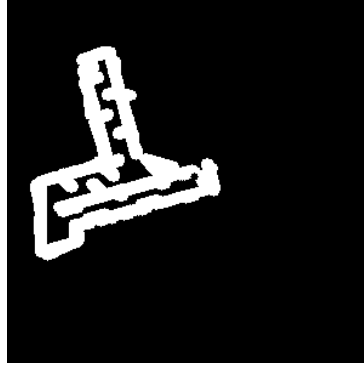


Figure 1: Inflated occupancy grid map with 4-pixel obstacle inflation used for A* path planning. Black regions represent free space, White regions represent inflated obstacles.

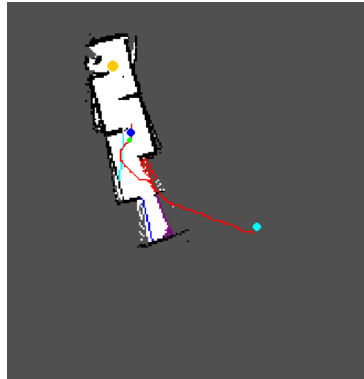


Figure 2: Frontier detection visualization. Colored regions indicate frontier clusters of varying sizes: blue (small), cyan (medium), yellow (large), and red (largest). The robot prioritizes frontier targets based on proximity to detected pillars and region size.

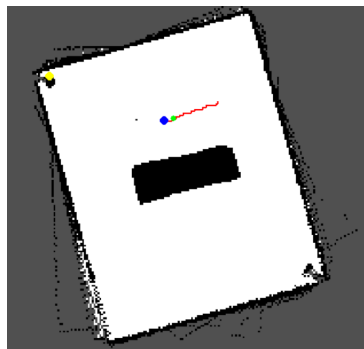


Figure 3: Occupancy grid visualization showing green carpet (poison) detection and permanent marking as black obstacle pixels. This marked region indicates a detected hazardous region projected from the camera onto the map. The blue dot represents the robot's current position, the small yellow dot marks the goal pillar (estimated), and the red line shows the planned path.